2025 International Ural Conference on Electrical Power Engineering (UralCon)

Anomaly Detection in Windows Event Logs for Ransomware Identification

Nikolay Nashivochnikov
Gazinformservice LLC
St. Petersburg, Russia
{cto, vitkova-l}@gaz-is.ru

Lidia Vitkova
Gazinformservice LLC
St. Petersburg, Russia
{cto, vitkova-l}@gaz-is.ru

Alexey Lukashin
Peter the Great St. Petersburg Polytechnic University
St. Petersburg, Russia
alexey.lukashin@spbstu.ru

Andrey Chechulin
International Digital Forenics Center
St. Petersburg Federal Research Center of the Russian Academy of Sciences
St. Petersburg, Russia
chechulin.a@spcras.ru

***Abstract*—This paper presents a machine learning architecture tailored for real-time anomaly detection within Windows operating system event logs, with a focus on identifying ransomware activity. The proposed approach is based on the DeepLog framework, employing a recurrent neural network of the Long Short-Term Memory type as its core component. Through training on sequences derived from normal system behavior, the model captures typical event patterns, enabling it to flag deviations that may signal malicious processes. The design allows for retraining when the set of monitored logs is updated, ensuring adaptability to evolving system conditions. This adaptability, combined with precise anomaly localization, makes the approach suitable for enhancing operational security and mitigating ransomware threats, as well as assisting digital forensic analysis by highlighting suspicious log sequences.**

Keywords—artificial intelligence, anomaly detection, cybersecurity, neural networks, digital forensics, ransomware

I. Introduction

Ransomware remains one of the most disruptive and damaging categories of cyber threats. Such malware encrypts user data and demands payment for decryption keys, while in some cases threatening to leak sensitive information. The popularity of these attacks among adversaries stems from the relative ease of implementation and high potential for financial gain. Traditional defense mechanisms, including signature-based antivirus solutions, often fail to detect new or previously unseen variants [1].

In this context, anomaly detection methods driven by machine learning have become increasingly significant. By monitoring deviations from established behavioral baselines, such systems can identify malicious activity in operating systems at an early stage, before critical damage occurs. Timely detection is essential. The analysis should be done almost in real time - otherwise the response (process termination or user alert) simply comes too late. This need for timely detection becomes even more critical in environments where automation and control of technological processes are involved, since disruptions in such systems can quickly propagate and cause large-scale consequences. A quite recent example is the ransomware attack on the Colonial Pipeline in 2021, where the compromise of operational systems led to the temporary shutdown of fuel distribution across the East Coast of the United States, demonstrating how malware incidents in industrial control environments can escalate into nationwide economic and social disruptions.

Beyond industrial and enterprise environments, the proposed approach is relevant for digital forensics. Windows Security Event Logs frequently serve as a key source of evidence during post-incident investigations, and anomaly detection methods can assist forensic analysts in reconstructing attacker activity, attributing malicious behavior, and identifying artifacts that may otherwise remain hidden.

Machine learning models for anomaly detection can be broadly categorized into offline and online approaches. Offline models, such as Principal Component Analysis applied to aggregated log statistics [2] or invariant mining for event sequence analysis [3], rely on retrospective data and are unsuitable for real-time mitigation. In contrast, the DeepLog framework [4] is designed for real-time monitoring and integrates multiple functional components for predicting both log key values and parameter values, as well as reconstructing workflow patterns.

For the purposes of this study, only the parameter value prediction component of DeepLog is employed. The choice is justified by the characteristics of the Windows Security Event Log:

- Each event contains a single key, with the variable content expressed entirely through its parameters.
- Process and thread identifiers are included directly in the log entries, making workflow reconstruction redundant.

Thus, the proposed method focuses exclusively on predicting parameter values within the security log to detect anomalies in Windows OS. This targeted adaptation allows for reduced model complexity while maintaining high detection accuracy in the context of ransomware activity.

The paper is organized as follows. Section II provides an overview of related works and summarizes the main approaches that form the basis of the study. Section III describes the architecture of the proposed model, and Section IV explains the processing of log entries required for

This study was partially supported by the Russian Science Foundation and the Saint Petersburg Science Foundation (grant No. 25-11-20028, https://rscf.ru/project/25-11-20028/) in SPC RAS.

979-8-3315-5323-4/25/$31.00 ©2025 IEEE

subsequent analysis. Section V sets out the anomaly detection procedure, and Section VI presents the experimental setup together with the obtained results. Section VII contains a discussion of the findings, including strengths and limitations, while Section VIII concludes the paper with key outcomes and possible directions for further research.

II. Related Works

Log-based anomaly detection has evolved rapidly over the past decade, moving from statistical baselines to deep neural architectures. Recent surveys provide a comprehensive map of the field, highlighting model families (RNNs, CNNs, Transformers), preprocessing pipelines, and evaluation pitfalls [5], [6]. A recurring conclusion is that generalization across systems and evolving log formats remains a core challenge, and that methods must be designed for streaming operation to be useful in security monitoring.

Early deep-learning systems such as DeepLog modelled event sequences with LSTMs and demonstrated effective online detection by learning normal patterns [4]. Successors explored robustness and representation learning beyond explicit templates. Transformer-based approaches, exemplified by LogBERT, apply self-supervised objectives to learn contextual dependencies without labels, improving detection across public datasets while mitigating exposure bias of autoregressive RNNs [7]. Hybrid designs combine attention with recurrent backbones or convolutional encoders to balance long-range context with efficient inference, reporting gains on cloud-scale traces [8]. Despite these advances, many Transformer variants incur higher serving costs and latency, which complicates strict real-time constraints in endpoint security.

A practical question is whether to parse logs into templates before modeling. Online parsers like Drain (and its industrialized variants) cluster raw messages into event templates in streaming mode, reducing vocabulary drift and sparsity for downstream models [9]. Tooling around Drain (e.g., Drain3) has become a de-facto baseline in production pipelines due to its stability and throughput. Nevertheless, parsing errors and evolving templates can introduce brittleness; several recent works therefore favor feature schemes that encode local structural relations (process/thread/user/domain) directly from structured sources (e.g., Windows Security Log fields), sidestepping heavy templating while preserving operational semantics.

Security research specific to Windows has increasingly leveraged operating-system telemetry for early ransomware detection. Frameworks built on Event Tracing for Windows (ETW) demonstrate that malware activity can be surfaced within seconds by correlating kernel and user-space events using hybrid rules and ML classifiers [10]. For incident response, practitioner reports show that Windows Event Logs carry discriminative signals across dozens of ransomware families, supporting both triage and retrospective analysis [11], which is directly relevant to forensic investigations. Parallel lines of work study supervised classifiers on ransomware corpora; while promising, such approaches can be sensitive to data drift and require continuous curation [12]. More recently, meta-learning and online adaptation have been proposed to maintain performance under distribution shifts in evolving systems, emphasizing semi-supervised learning and conditional updates in streaming settings [13]. Empirical assessments further caution that model rankings vary with data grouping, noise, and early-detection criteria, urging careful protocol design for fair comparisons [14].

Relative to the above, our work targets real-time detection on Windows Security Event Logs with strict latency and low memory footprint. We adopt a compact BiLSTM backbone but diverge from classic DeepLog:

- Parameter-only DeepLog adaptation. We employ exclusively the parameter value prediction component and omit key-sequence and workflow reconstruction. This is justified by the structure of Windows Security Log entries, which already carry process/thread identifiers and task context, allowing us to avoid explicit workflow modeling without losing discriminative power.
- Semantics-aware feature design. Instead of template mining, we encode equality indicators over security-relevant fields (e.g., ProcessID, ThreadID, user/domain/process names, Task, LogonID) and a stabilized elapsed-time feature. This representation is resilient to template drift and preserves local controlflow and identity continuity that are characteristic of ransomware activity.
- Adaptive decision logic for streaming. We estimate the error distribution via kernel density estimation to derive an α-percentile threshold per deployment, then apply a sliding-window anomaly ratio to suppress spurious spikes. This two-stage decision rule aligns with operational needs (few false alarms, rapid confirmation) while keeping inference under 1 ms per event on commodity GPUs.

In combination, these design choices close the gap between high-throughput operational monitoring and the accuracy of modern sequence models. Our experiments on Windows 11 logs and controlled ransomware executions show that the approach detects encryption onsets within roughly one minute while keeping benign activities (e.g., installers, scheduled archiving) below alerting thresholds.

Surveys and recent systems underline the tension between accuracy, robustness to evolving logs, and deployment constraints. Transformer-based detectors push accuracy but often at a higher serving cost; Windows-specific frameworks using ETW demonstrate timeliness but typically rely on handcrafted rules or supervised classifiers. Our contribution is a middle path: a compact, streaming-friendly sequence model with semantic features from Windows Security Logs and an adaptive, distribution-aware decision layer that reduces false positives without template mining or heavy context encoders.

III. Model Architecture

The proposed architecture is built upon a two-layer bidirectional Long Short-Term Memory (BiLSTM) recurrent

neural network. This design enables the model to capture temporal dependencies in both forward and backward directions within sequences of Windows Security Event Log entries. By learning on event sequences associated with normal system activity, the network develops an internal representation of expected parameter patterns and can flag deviations indicative of anomalous behavior [15].

Each log entry is transformed into a feature vector comprising:

- One-hot encoded eventId;
- Normalized cubic root of elapsed time between consecutive events, bounded in magnitude by 1.5;
- Binary and ternary indicators (0, 0.5, 1) reflecting equality or absence of values in key fields such as ThreadID, ProcessID, Task, SubjectUserName, SubjectDomainName, SubjectProcessName, and SubjectLogonID.

For improved detection reliability, the model applies a kernel density estimation (KDE) method to establish an adaptive anomaly threshold. Additionally, a sliding window mechanism is employed to calculate the proportion of anomalous predictions within a recent segment of activity. This approach reduces false positives by considering anomalies in their operational context.

Training is conducted using mini-batch gradient descent with the Adam optimizer. The loss function is based on the mean squared error (MSE), computed separately for categorical (one-hot) and continuous features, with weighted aggregation to reflect their relative importance. The architecture's compact design ensures low computational overhead, enabling sub-millisecond inference per log entry and making it suitable for real-time security monitoring in production environments.

IV. Log Entry Processing

To train the model and subsequently detect anomalies, each record from the Windows Security Event Log must be converted into a structured numerical representation. In this study, each log entry is mapped to a feature vector containing the following components:

- eventId encoded using a one-hot representation;
- Elapsed time between consecutive records (EventRecordId-ordered), transformed via the normalized cubic root function and constrained to an absolute value of 1.5;
- Equality indicators (0, 0.5, or 1) for ThreadID,
- ProcessID, Task, SubjectUserName, SubjectDomainName, SubjectProcessName, and SubjectLogonID, comparing the current and previous events.

The equality indicators are defined as follows: 0 if the fields differ, 1 if they match, and 0.5 if the field value is absent in at least one of the compared events.

The feature set can be extended to capture more complex dependencies by introducing indicators that compare the current event with entries occurring several steps back in the log. This enhancement is particularly relevant in multi-threaded or multi-process environments where event interleaving is frequent.

By adopting such structured preprocessing, the model gains the ability to learn intricate temporal and relational patterns present in system activity, which is critical for distinguishing legitimate behavior from malicious anomalies.

V. Anomaly Detection

The anomaly detection process is based on predicting the next event parameter vector from a sliding context window and measuring the deviation between the predicted and actual values. The procedure consists of the following steps:

1. Data preparation: Collect a log dataset representing normal system activity. Sort the records by EventRecordId and split them into training and validation sets.
2. Sequence generation: Partition each dataset into fixed size subsequences of length W, using a sliding window with a stride of 1. The entry immediately following each subsequence serves as the prediction target.
3. Model training: Train the BiLSTM model on the training set using mini batches of size B with the Adam optimizer. The loss function is the mean squared error (MSE), calculated separately for one-hot encoded and continuous features, then combined via weighted aggregation.
4. Error distribution estimation: Apply kernel density estimation to the validation set prediction errors to obtain the probability density function p(e). Determine the anomaly threshold as the α-percentile of the estimated distribution.
5. Anomaly detection in operation: For each incoming event, predict its parameter vector, compute the MSE with respect to the actual vector, and flag the record as anomalous if the error exceeds the threshold.

Since even normal activity may occasionally exceed the threshold, the model incorporates a smoothing mechanism based on a sliding window of size S over recent events. If the proportion of anomalies within this window exceeds a predefined ratio β, the system flags the situation as an anomalous activity episode. This reduces false positives by requiring consistent anomaly patterns before triggering an alert.

This method ensures that ransomware-induced behavior, characterized by rapid and systematic file operations, is detected in real time while maintaining robustness against benign activity spikes.

VI. Experiments

The model was implemented using the PyTorch machine learning framework. Experimental evaluation was conducted

on Windows 11 Security Event Logs collected over three days of normal user activity, including web browsing, file management, and application usage [16].

A. Experimental Setup

The key hyperparameters and dataset characteristics are as follows:

- Training set size: 2,662,780 records;
- Validation set size: 295,000 records;
- Context window size (W): 100 events;
- Mini-batch size (B): 64;
- Optimizer: Adam;
- Learning rate: empirically tuned;
- Anomaly detection threshold: 80th percentile of validation error distribution;
- Sliding window sizes (S): 1000, 2000, and 40,000 events;
- Activity anomaly ratio (β): 0.67.

The training and validation datasets were sorted by EventRecordId to preserve temporal order. The model parameters were optimized to minimize false positives while retaining high sensitivity to ransomware-induced anomalies.

B. Training Process

The BiLSTM network contained 2,271,809 trainable parameters. Training was performed on two Nvidia V100 GPUs, requiring 2.2 GB of memory during training and only 26 MB during inference. The total training time was 8 hours and 44 minutes. The dependence of the average training loss on the gradient descent step is illustrated in Fig. 1. Inference time per event was less than 1 ms, confirming the model's suitability for real-time deployment. The dependence of the average validation loss on the gradient descent step is shown in Fig. 2.

C. Results

Kernel density estimation of validation errors revealed a well-defined anomaly zone. Selecting a sliding window size of S = 40,000 events and β = 0.67 provided a balance between detection sensitivity and false positive suppression. The kernel density estimate of validation errors, with the anomaly zone highlighted, is presented in Fig. 3.

Controlled ransomware execution experiments were performed with two malware samples: Jigsaw and Vipasana. The anomaly ratio during the execution of the Jigsaw ransomware is shown in Fig. 4. The anomaly ratio during the execution of the Vipasana ransomware is illustrated in Fig. 5. In both cases, the system successfully detected anomalous activity within one minute of encryption onset, while maintaining a low false alarm rate during normal operations such as software installation or scheduled file archiving. The anomaly ratio during Notepad++ installation is depicted in Fig. 6. The anomaly ratio observed during hourly file archiving is shown in Fig. 7. The anomaly ratio over one hour of normal user activity is presented in Fig. 8.

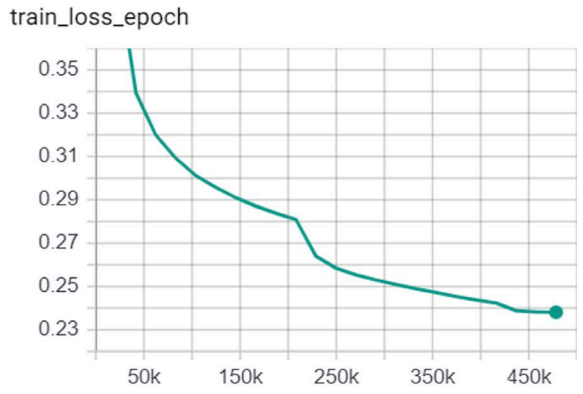


Fig. 1. Training loss vs. optimization step on the training set.

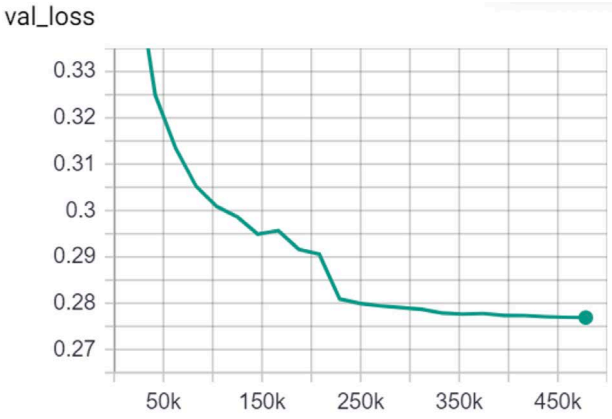


Fig. 2. Validation loss vs. optimization step on the validation set.

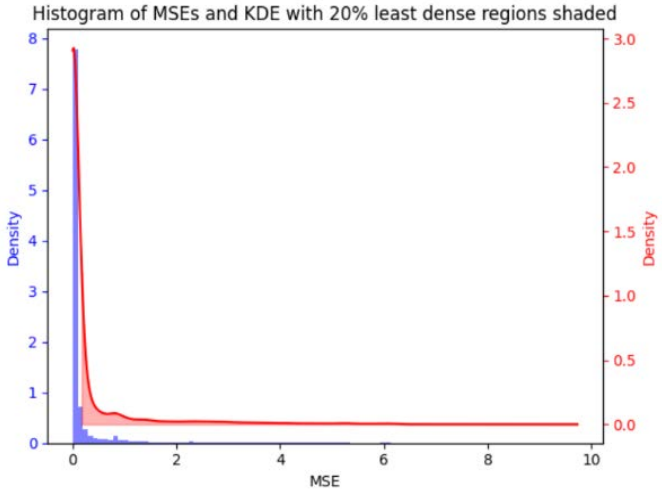


Fig. 3. Kernel density estimate of prediction errors on the validation set; the shaded region denotes the anomaly zone (threshold at the chosen percentile).

The findings confirm that the proposed configuration can identify ransomware in real time and is generalized to other types of malicious software that produce distinctive patterns in

security event logs, making them valuable not only for defense but also for forensic reconstruction of incidents.

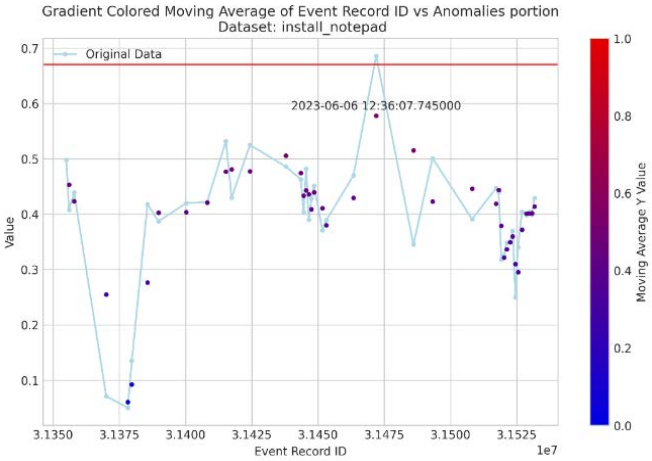


Fig. 4. Anomaly ratio over EventRecordId during execution of the Jigsaw ransomware sample.

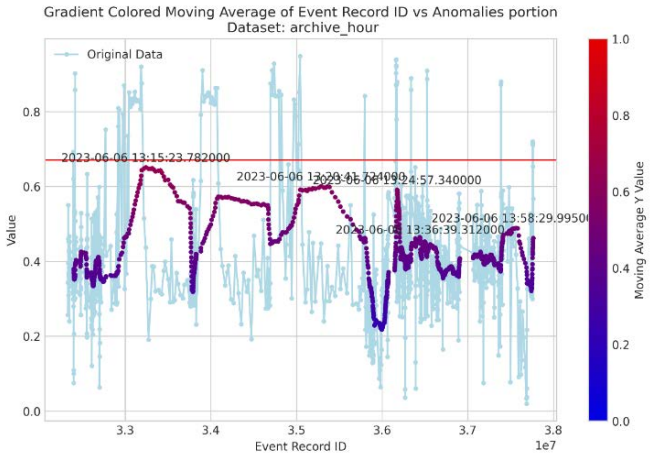


Fig. 5. Anomaly ratio over EventRecordId during execution of the Vipasana ransomware sample.

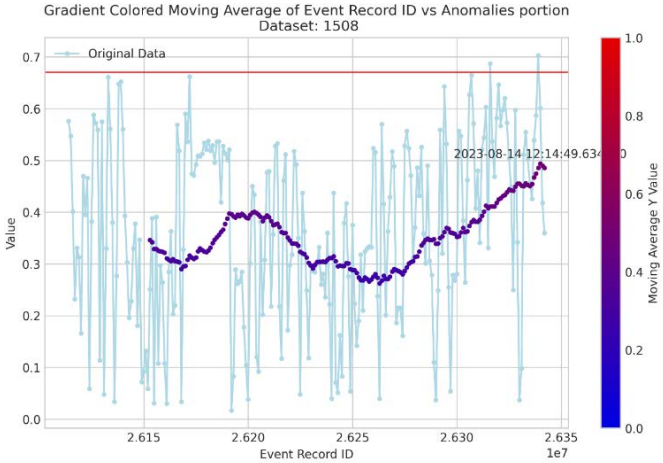


Fig. 6. Anomaly ratio over EventRecordId during Notepad++ installation (comparison of window sizes S).

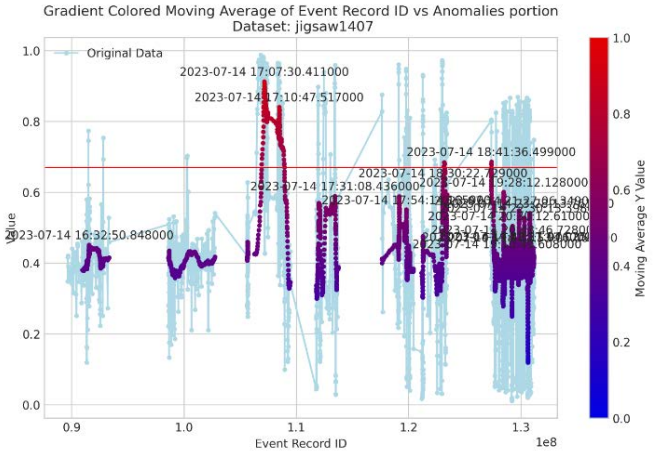


Fig. 7. Anomaly ratio over EventRecordId during hourly file archiving.

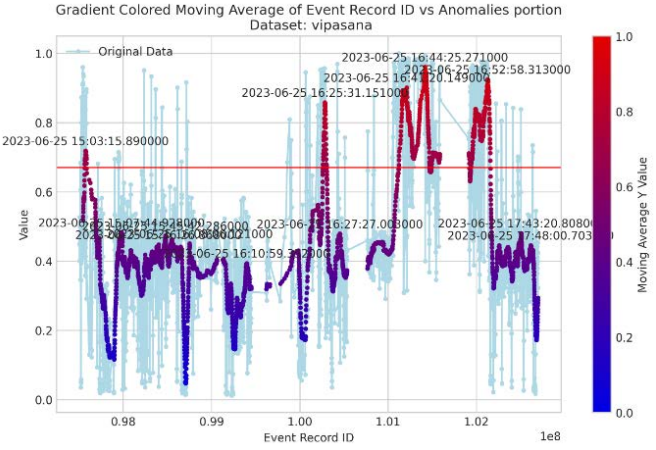


Fig. 8. Anomaly ratio over EventRecordId during one hour of normal user activity.

VII. Discussion

The experimental results show that the proposed BiLSTM-based architecture, enhanced with kernel density estimation and a sliding-window anomaly ratio, can detect ransomware activity in Windows Security Event Logs within roughly one minute after the onset of file encryption. This latency is sufficient to initiate rapid countermeasures, such as process termination or network isolation, before large-scale data loss occurs. The chosen configuration parameters ensured a good balance between sensitivity and stability: larger sliding windows substantially reduced false alarms during benign activities, like application installation, yet still allowed timely recognition of sustained malicious processes. The anomaly ratio threshold of 0.67, empirically determined, appeared to be a reasonable compromise, avoiding excessive alerting while preserving responsiveness.

An important design decision was to rely exclusively on the parameter value prediction component of DeepLog. In the case of Windows Security Logs, essential contextual information, e.g. process and thread identifiers, user and domain names, task attributes, is already embedded in the parameters, making

additional key-sequence modeling unnecessary. This allowed us to simplify the architecture without losing discriminative capability and at the same time achieve low memory usage and submillisecond processing time per event, which is critical for continuous, real-time deployment.

The approach has its limits. Its accuracy strongly depends on how representative the training baseline is - if system activity changes too much, false positives will grow until the model is retrained. Routine shifts like OS updates or new software may also alter log patterns and force a threshold adjustment. The method only works when attacks leave traces in the security log; memory-only malware or tampered logs may go unnoticed, so extra evidence sources remain important in forensic work. Large sliding windows make detection steadier, but at the cost of some delay in spotting very short attacks. Finally, adapting the feature set to other operating systems is possible, but requires careful mapping of fields and semantics.

Despite these constraints, the experiments indicate that the model meets its main objectives: it delivers fast and accurate detection of ransomware behavior, operates efficiently on modest hardware, and maintains a low false positive rate in realistic usage scenarios. The identified limitations are specific, well-understood, and can be addressed in future work through incremental improvements such as adaptive thresholds, online learning, or the integration of complementary telemetry sources. By anticipating these challenges and outlining ways to overcome them, the practical relevance and credibility of the proposed approach are reinforced.

VIII. Conclusion

This work presents a machine learning approach for real-time anomaly detection in Windows Security Event Logs, specifically targeting ransomware activity. The proposed model is based on the parameter prediction component of the DeepLog framework, adapted to the structural characteristics of Windows event logs.

Experimental results demonstrate that the BiLSTM-based architecture, combined with kernel density estimation and sliding window analysis, can detect ransomware-induced anomalies within one minute of attack initiation while maintaining a low false positive rate. The selected configuration parameters provide a practical balance between sensitivity and operational stability, making the method suitable for deployment in real world security monitoring systems.

Future work will focus on extending the model to operate in a dual mode: as an anomaly detector for unknown threats and as a classifier trained specifically on known ransomware behaviors. Such a hybrid approach would enable the system to detect novel attacks while improving the accuracy of identifying previously encountered threats. Additionally, adapting the parameter vector structure for logs from other operating systems would broaden the applicability of the method beyond the Windows environment. In addition to real-time protection, the method has forensic applications, supporting investigators in evidence extraction and reconstruction of attack timelines based on anomalous log patterns [17].

References

[1] I. Kotenko, D. Levshun, and A. Chechulin, “Event correlation in the integrated cyber-physical security system,” in Proc. XIX IEEE Int. Conf. Soft Computing and Measurements (SCM), 2016, pp. 484–486.

[2] W. Xu, L. Huang, A. Fox, D. Patterson, and M. Jordan, “Detecting large-scale system problems by mining console logs,” in Proc. ICML10, 2010, pp. 37–46, doi: 10.1145/1629575.1629587.

[3] J.-G. Lou, Q. Fu, S. Yang, J. Li, and B. Wu, “Mining invariants from console logs for system problem detection,” in Proc. 16th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), 2010, pp. 613–622, doi: 10.1145/1835804.1835883.

[4] M. Du, F. Li, G. Zheng, and V. Srikumar, “DeepLog: Anomaly detection and diagnosis from system logs through deep learning,” in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2017, pp. 1285–1298, doi: 10.1145/3133956.3134015.

[5] M. Landauer, F. Skopik, M. Wurzenberger, and A. Rauber, “System log clustering approaches for cyber security applications: A survey,” Computers & Security, vol. 92, 2020, art. no. 101739, doi: 10.1016/j.cose.2020.101739.

[6] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, “Deep learning for anomaly detection in log data: A survey,” Machine Learning with Applications, vol. 12, 2023, art. no. 100470, doi: 10.1016/j.mlwa.2023.100470.

[7] Z. H. Guo, S. Yuan, and X. Wu, “LogBERT: Log anomaly detection via BERT,” in Proc. International Joint Conference on Neural Networks (IJCNN), 2021, pp. 1–8, doi: 10.1109/IJCNN52387.2021.9534113.

[8] X. Ou and J. Liu, “Log anomaly detection method based on hybrid Transformer-BiLSTM models,” in Proc. IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C), Dec. 2022, pp. 762–763, doi: 10.1109/QRS-C57518.2022.00123.

[9] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, “Drain: An Online Log Parsing Approach with Fixed Depth Tree,” in Proc. IEEE Int. Conf. Web Services (ICWS), 2017, pp. 33–40. doi: 10.1109/ICWS.2017.11.

[10] M. Adnan and A. Jalil, “RansomGuard: A framework for proactive detection and mitigation of cryptographic windows ransomware,” in Journal of Computer Virology and Hacking Techniques, vol. 20, pp. 867–884, 2024.

[11] SANS Institute, “Triaging Windows Event Logs for Ransomware Investigations,” Whitepaper, 2022. [Online]. Available: https://www.sans.org/

[12] M. Ring, D. Schlör, S. Wunderlich, D. Landes, and A. Hotho, “Malware detection on Windows audit logs using LSTMs,” Computers & Security, vol. 109, 2021, art. no. 102389, doi: 10.1016/j.cose.2021.102389.

[13] J. Tian, M. Li, Z. Wang, L. Chen, J. Qin, and R. Zhang, “OMLog: Online log anomaly detection for evolving systems with meta-learning,” arXiv preprint, arXiv:2410.16612, 2024. [Online]. Available: https://arxiv.org/pdf/2410.16612

[14] S. Ali, C. Boufaied, D. Bianculli, P. Branco, and L. Briand, “A comprehensive study of machine learning techniques for log-based anomaly detection,” arXiv preprint, arXiv:2307.16714v5 [cs.SE], 2025. [Online]. Available: https://arxiv.org/pdf/2307.16714

[15] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[16] A. Chechulin and M. Gorda, “Cybercrime investigation model,” in Proc. 16th Int. Conf. Communication Systems and NETworkS (COMSNETS), Cyber-Security and Privacy Workshop, 2024, pp. 37–42, doi: 10.1109/COMSNETS59351.2024.10427215.

[17] M. Gorda and A. Chechulin, “Evidences extraction and analysis challenges,” in Proc. 2024 International Russian Smart Industry Conference (SmartIndustryCon), 2024, pp. 715–719, doi: 10.1109/SmartIndustryCon61328.2024.10515824.